Ensaio de 2.000 req/s em 10 minutos

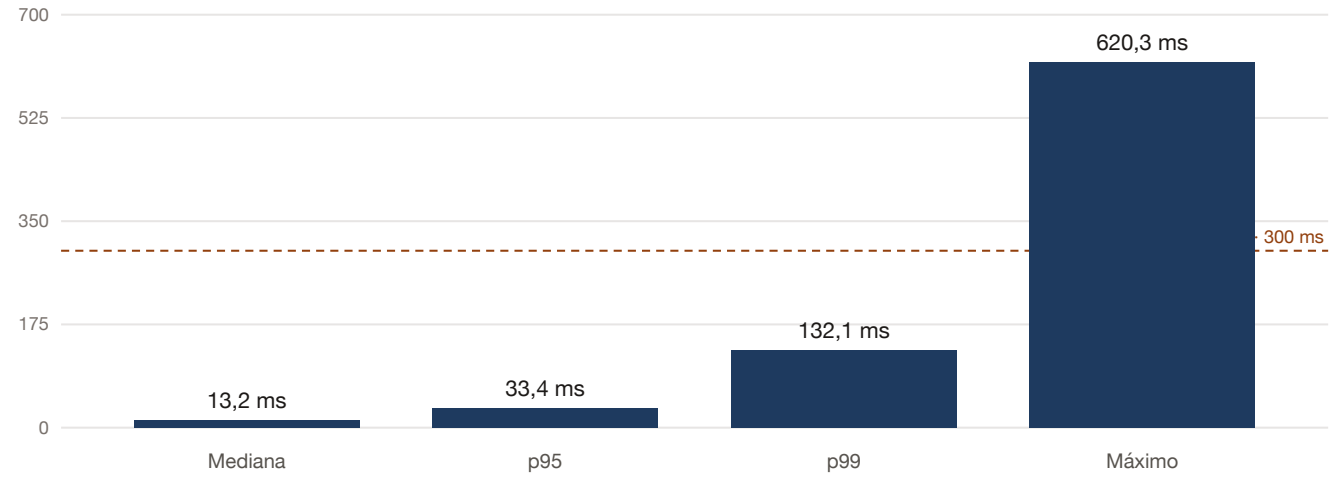
Conta quente 1, modo misto, duas APIs e PostgreSQL 16. Volume recriado, 1.000.000 de clientes semeados, depois k6 com chegada aberta. 29 de setembro de 2026, 15:19–15:29, horário de Brasília.

> **A meta de vazão passou no p99 e falhou nos erros.**
> O p99 ficou em 132 ms, abaixo de 300 ms, e nenhuma iteração foi descartada. Três das 1.200.013 requisições não voltaram HTTP 200. O k6 encerrou com código 99 por causa desses três casos. Nenhuma delas gerou operação no banco.

132 ms	1.999,9/s	1.200.010	3
p99 HTTP ponta a ponta	Taxa concluída	Aprovações gravadas	Respostas fora de HTTP 200

Latência HTTP do ensaio

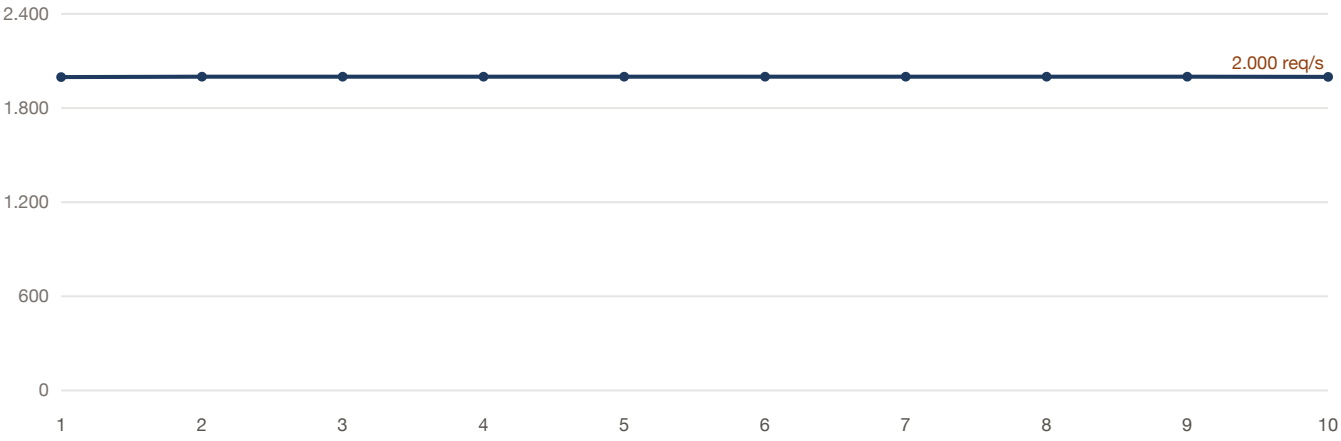
Mediana, p95, p99 e máximo de http_req_duration. A linha tracejada é a meta de p99 abaixo de 300 ms. Eixo vertical em milissegundos.



Fonte: reports/load-reset-10m.json · k6 · 10m00s · localhost:8080 e localhost:8081

Vazão sustentada

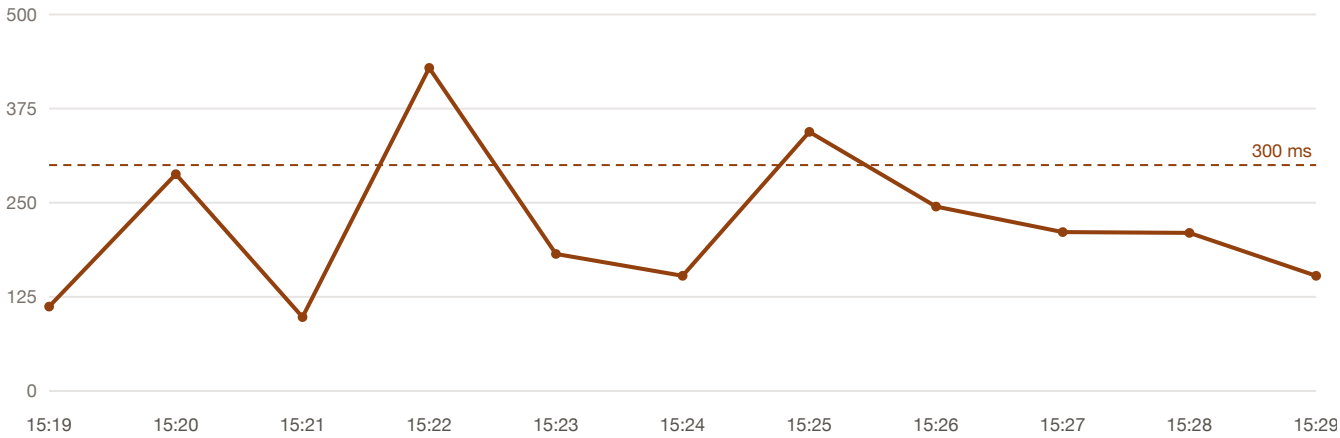
Requisições concluídas por segundo em cada minuto do relógio do k6. A linha tracejada é a taxa pedida, 2.000 req/s. Eixo vertical: req/s, começando em zero.



Eixo horizontal: minuto do ensaio, de 1 a 10. Fonte: progresso do k6, 15:19–15:29.

Maior commit de lote

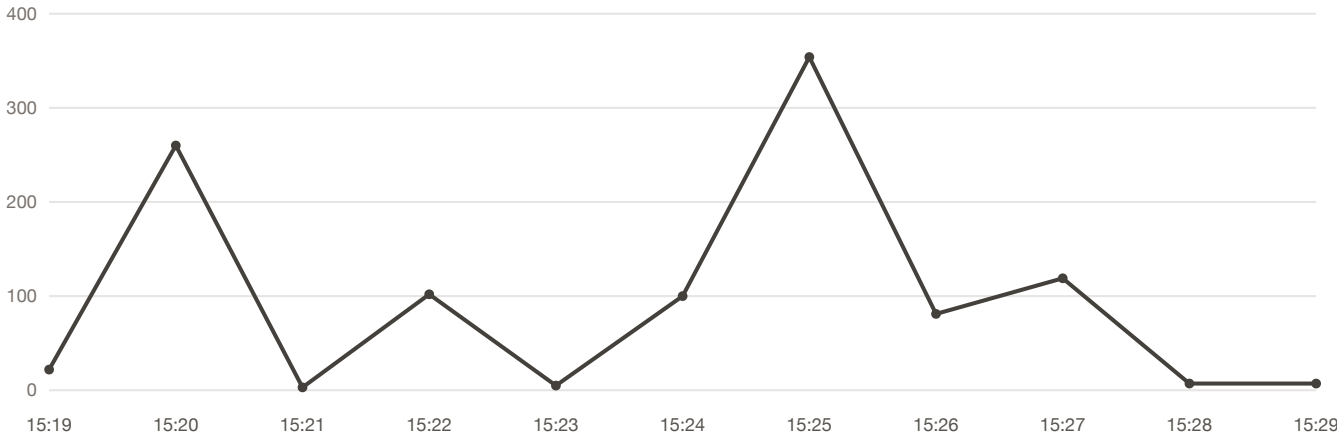
Máximo de max_batch_ms entre as duas APIs, em cada minuto. Eixo horizontal: hora local. Eixo vertical: milissegundos. A linha tracejada é 300 ms.



15:19 e 15:29 são minutos parciais. Fonte: log queue stats.

Profundidade da fila

Maior depth no intervalo de 5 s. Cada fila cabe 4.096 pedidos. Eixo horizontal: hora local. Eixo vertical: pedidos.



Fonte: log queue stats das duas APIs.

Critério do ensaio saudável

Critério	Meta	Medido	Resultado
p99 HTTP	< 300 ms	132,1 ms	**Passou**
Erros técnicos	0	3	**Falhou**
Iterações descartadas	0	0	**Passou**
Fila cheia	0	0	**Passou**
Timeout de lote	0	0	**Passou**

As três falhas entraram no contador de outro status do script: nem HTTP 200 nem HTTP 503. Os logs das duas APIs fecharam com batch_fail=0, queue_full=0 e timeout_waiting=0. O banco tem 1.200.010 operações, o mesmo número de HTTP 200.

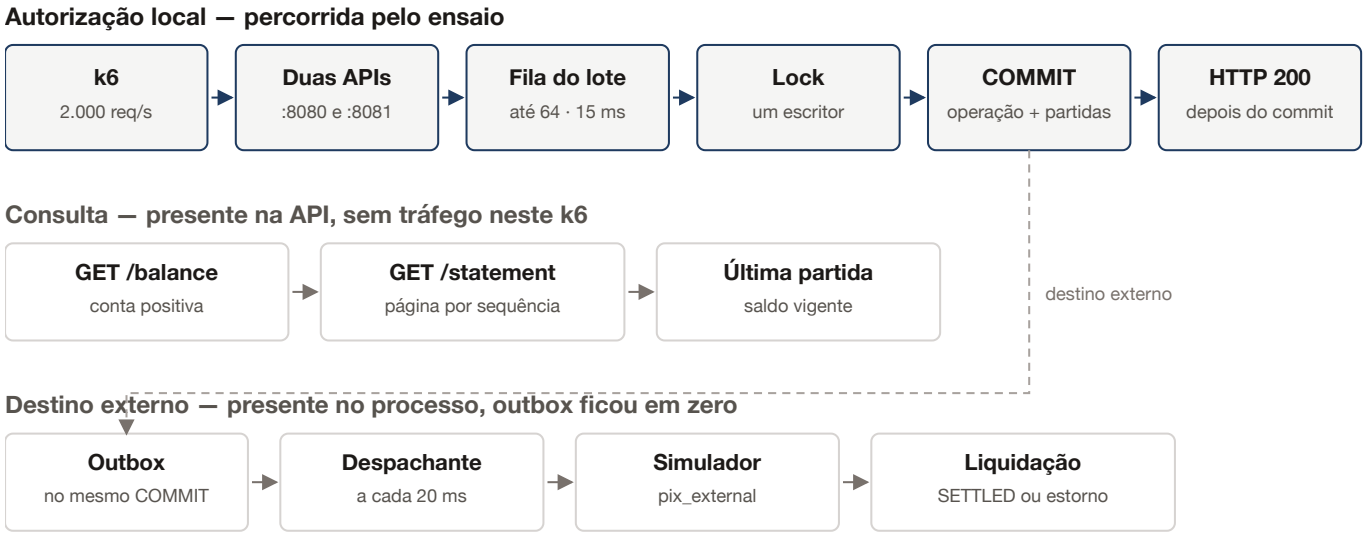
O que ficou gravado

Conferência no PostgreSQL	Valor
Contas (1.000.000 clientes + 32 controles)	1.000.032
Operações, todas APPROVED	1.200.010
DECLINED	0
Partidas (duas por aprovação)	2.400.020
Partidas da conta 1	1.200.010
Última sequência da conta 1	1.200.010
Saldo final da conta 1	1.000.000.000.000
Menor saldo visto na conta 1	999.999.999.935
Prefixos negativos na conta 1	0
Eventos na outbox	0

Consultas SQL logo após o k6, banco pix. O saldo está em centavos. Cada transferência movia 1 centavo, metade entrando e metade saindo da conta 1, então o saldo final voltou ao seed. A linha de accounts da conta 1 continua com last_sequence 0: o caminho do lote grava a sequência só em entries.

Fluxo do autorizador

O ensaio percorreu só a faixa de cima: transferência local, conta 1 contra um par, valor de 1 centavo. As faixas de baixo existem no processo e ficaram paradas nesta execução. O traço tracejado sai do COMMIT quando o destino é externo.



Lote 64, janela de 15 ms, lock consultivo e prazo de commit de 1,2 s. fsync e synchronous_commit ligados.

Camadas do processo

Peça	O que faz neste desenho
k6 · scripts/load.js	Chegada aberta. Alterna crédito e débito da conta 1 e reparte as chamadas entre as duas portas.
cmd/pix	Sobe a API, aplica migração e, com APP_MODE=simulator, vira o destinatário externo.
internal/httpapi	Autentica a chave, decodifica o JSON e espera a resposta do lote. O despachante da outbox mora aqui.
internal/ledger	Decide cada débito em ordem, com saldo intermediário, sem I/O.
internal/postgres	Fila em memória, lock consultivo, COMMIT de operations e entries.
PostgreSQL 16	Autoridade do saldo. fsync e synchronous_commit ligados. shared_buffers de 512 MB.
Simulador	Banco pix_external. Um efeito por ID. O prefixo reject: no destinatário vira rejeição definitiva.

Escala: o que este ensaio mostra

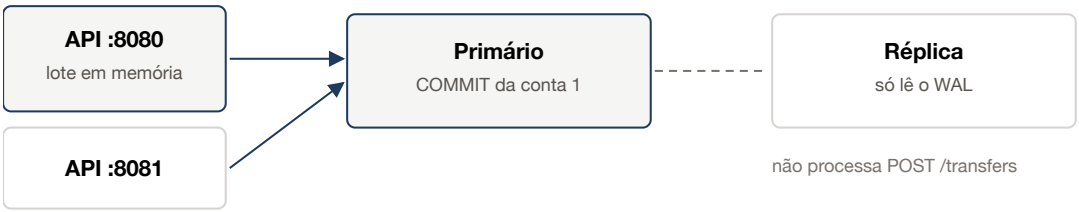
A recepção HTTP já corre ao lado do banco: cada POST entra na fila e espera o canal de resposta. O que não paraleliza é o próximo COMMIT. Um único goroutine por API drena a fila; enquanto o lote está no fsync, pedidos novos só ocupam o canal (4.096 lugares). Neste ensaio a profundidade chegou a 354, queue_full ficou em 0 e o p99 acompanhou os commits lentos (até 429 ms), não a CPU da API.

Escalar horizontalmente aqui é partir o razão por conta, não multiplicar o processo HTTP. Mais API na frente do mesmo primário não aumenta a vazão da conta quente. O ensaio já rodou com duas instâncias: as duas esperam o lock 937513 e um COMMIT com fsync.

Duas APIs, um primário, réplica

O lote não escolhe uma réplica. Toda autorização grava no primário.

O lote das duas APIs grava no mesmo primário



As duas APIs deste Compose apontam para o mesmo DATABASE_URL. Réplica de leitura não existia neste ensaio; o desenho mostra o papel dela se existisse.

Peça	O que faz	O que não faz
API :8080 e :8081	Aceitam HTTP. Se uma cai, a outra segue. As duas mandam SQL para o mesmo Postgres.	Não dobram a vazão da conta 1. Os lotes entram no mesmo lock 937513.
Primário PostgreSQL	Único lugar que faz COMMIT de operations e entries. O lote sempre grava aqui.	Não despacha “para uma réplica”. Não há segundo escritor da conta 1.
Réplica de leitura	Copia o WAL já commitado. Serve GET se o atraso for aceitável. Se o primário morre, pode ser promovida (failover).	Não processa lote. Não disputa COMMIT. Não aumenta 2.000 req/s na conta quente.

Réplica não é segundo primário. Dois bancos gravando a conta 1 ao mesmo tempo recriam a disputa que o lote existe para evitar. Exemplo: saldo 100, dois créditos de 1. Um escritor junta os dois, decide 101 e 102 e faz um fsync. Dois primários leem os dois “última partida = 100”, tentam a próxima sequência, dois fsync, cauda alta. Foi o que aconteceu quando as duas APIs reescreviam a linha da conta 1; o lock único deixou um lote no disco por vez.

O que seria um shard

Shard é um pedaço autônomo do razão: um conjunto de contas, o Postgres que as guarda e o escritor que avança o saldo delas. Não é réplica da API. A API continua stateless. Conta 1 mora num pedaço; outras contas em outro. Cada shard tem o próprio accounts, operations e entries daquelas contas, e um COMMIT próprio. Dois shards não atualizam a mesma linha.

O protótipo já usa a ideia em miniatura: as 32 contas técnicas de saída, escolhidas por hash estável da origem, para o Pix externo não brigar por uma linha só. O próximo passo seria o mesmo critério para clientes: hash(account_id) → qual banco.

Transferência **dentro** do shard é o fluxo deste ensaio: lote, duas partidas, COMMIT, depois HTTP 200. Transferência **entre** shards não cabe nesse COMMIT: débito num disco e crédito noutro, como o destino externo já faz com outbox e PENDING. A conta que recebe 2.000 req/s não se parte: ela inteira vai para um shard só. Espelhar essa conta em dois primários recria a disputa acima.

> **Neste ensaio de 10 minutos, shard não melhoraria o resultado.**
> Os 2.000 req/s tocam a conta 1. Um shard por conta mandaria tudo ao mesmo pedaço. O COMMIT, o fsync e o escritor único continuariam iguais. O p99 de 132 ms não cairia. Shard ajuda o total quando o tráfego se espalha por contas que não se cruzam — e só depois de o lock deixar de ser um para o banco inteiro (hoje o 937513 serializa até contas frias).

Se a meta for 4.000 req/s

Não se infere do p99 de 132 ms. Medir 2.500, 3.000 e 4.000 no mesmo cenário misto. Manter fsync; desligar não é “dar suporte”.

Cenário	O que falta
Conta 1 a 4.000/s	Dobro deste ensaio no mesmo caderno. Com lote 64 seriam ~63 COMMIT/s (cerca de 16 ms por gravação). O lote já chegou a 429 ms: sem mais pedidos por fsync, um escritor só da conta 1 e WAL mais rápido, a fila de 4.096 enche e volta erro técnico. Réplica e segundo primário da conta 1 não resolvem. HTTP pode repetir; quem commita a conta 1 deve ser um.
4.000 no total, contas que quase não se cruzam	O teto atual é o lock 937513 (um COMMIT no banco inteiro). Primeiro travar só as contas do lote; depois shards. Cada fatia aguenta o próprio lote. Quem sozinha chegar a 4.000 continua um shard só dela.

O contrato não muda em nenhum desenho: APPROVED depois do COMMIT, um efeito por ID, duas partidas por aprovação, saldo nunca negativo em todo prefixo, erro técnico nunca vira DECLINED.

> **Limite deste relatório.** Estes números são o ensaio misto de 10 minutos depois do reset. Consulta de saldo, extrato, pagamento externo, contas frias, fan-in isolado, fan-out isolado e o auditor independente não rodaram nesta execução. A discussão de escala e de 4.000 req/s não foi medida.